



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

ARDUINO PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A IoT & Edge

TOTAL: 10 QUESTIONS

Q1 What is Arduino and what problem in IoT & Edge does it solve? [What Model](#)

Q6 How does Arduino handle reliability and high availability? [Reliability](#)

Q2 What are the core components or concepts in Arduino? [Architecture](#)

Q7 What observability does Arduino provide out of the box? [Lifecycle](#)

Q3 How does Arduino compare to its main alternatives? [Configuration](#)

Q8 What are common performance bottlenecks in Arduino? [Compliance](#)

Q4 How do you get started with Arduino for a new project? [Hardening](#)

Q9 What security best practices apply when running Arduino? [Testing](#)

Q5 What configuration options most affect Arduino's behavior in production? [Deployment](#)

Q10 What mistakes do teams commonly make when adopting Arduino? [Tuning](#)



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Arduino is a tool or platform that

Mitigation

Arduino is a tool or platform that automates or simplifies a specific concern in the IoT & Edge domain, reducing manual effort and operational complexity for engineering teams.

A6 Arduino provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

Configuration, availability

A2 Arduino has key building blocks that work

Primitives

Arduino has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A7 Arduino exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

Configuration, mitigation

A3 Arduino trades off simplicity against feature richness

Hardening

Arduino trades off simplicity against feature richness differently than its alternatives. Choose Arduino when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A8 Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

Configuration, performance

A4 Install Arduino via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

Gotchas

A9 Run Arduino with the principle of least

Assessment

Run Arduino with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A5 Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

Configuration, mitigation

A10 Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.

Configuration, mitigation